설비 고장을 예측하는 AI 데이터 분석

Pandas 집계·기초 통계

빈도와 그룹 집계

posco

CONTENTS

목차

01 빈도 집계 — 값별 개수와 비율, 구간으로 묶어 세기

02 그룹 집계 — 분할•적용•결합, 여러 기준으로 나눠 보기

03 종합 실습 —빈도와 그룹을 이어 보기

posco

이 단원의 실데이터 —유압 시스템 상태 감시

공정 — 유압 시스템 상태 감시

유압 시스템이 60초 부하 사이클을 반복하며 압력·온도·진동을 측정한다.
각 사이클에는 냉각기·밸브·펌프의 상태와 운전 부하가 함께 기록된다.

핵심 범주와 연속 변수

범주형 = 냉각기상태(정상/저하/고장) · 운전부하(고부하/저부하) · 밸브상태(정상/경미/지연/심각). 연속형 = 온도·진동·압력·냉각효율.
result 는 그 사이클의 정상/고장 판정.

정상과 이상

냉각기가 정상이면 온도 약 36℃, 저하 45℃, 고장이면 55℃로 상태별로 뚜렷이 갈린다.
그래서 상태(범주)로 묶으면 그룹 간 차이가 선명하게 드러난다.

왜 그룹 집계인가

냉각기·밸브·부하 같은 범주로 데이터를 나눠(groupby) 그룹별 평균·건수를 비교하면, 어느 상태에서 온도·진동이 위험한지 한눈에 파악할 수 있다.

01

빈도 집계

값별 개수와 비율 · 수치형을 구간으로 묶어 세기

행 하나하나 보기의 한계

120행도 한 줄씩 읽기는 어렵다 — 요약해 전체를 한눈에

행을 하나씩 읽는 대신 요약해 전체 파악

사이클 120개 × 열 8개 = 값 960개 — 눈으로 읽어 구성을 파악하기는 어려움

집계란 여러 값을 한 숫자로 압축

집계로 답할 수 있는 질문

빈도 · 그룹 비교 · 관계 — 오늘부터 배울 세 가지 핵심 질문

01 · 빈도

어떤 값이 가장 많은가

「요즘 밸브 쪽이 이상하다」는 말의 첫 검증 단계

02 · 그룹 비교

그룹마다 평균이 다른가

냉각기 상태별 · 밸브 상태별 차이를 보는 분석의 기본 단위

03 · 관계

두 값이 함께 움직이는가

온도와 냉각효율, 두 측정값이 함께 움직이는지 확인

이번 파트에서는 빈도·그룹 집계·상관관계 — 설비 데이터 자가 파악의 도구상자

범주형과 수치형 데이터

셀 수 있는 데이터 구분이 먼저 — 설비 데이터는 두 종류가 섞여 있음

01 · 수치형

숫자로 측정, 평균·합계 가능

온도 45.3도 — 더하거나 평균 낼 수 있는 측정값

02 · 범주형

종류로 구분, 빈도 집계 대상

정상 · 저하 · 고장 — 종류를 나타내는 값

03 · 빈도란

각 종류가 몇 번 나오는지

범주형에서 빛을 발하는 첫 번째 집계

숫자가 들어 있어도 이름 대신 쓰는 값이면 범주형 — 쓰임이 구분 기준

value_counts로 개수 세기

한 열에 어떤 값이 몇 번씩 나오는지 자동으로 세는 도구

01 • 이름의 뜻

value(값)를 counts(센다) —이름 그대로의 동작

02 • 사용법

세고 싶은 열을 고른 뒤 점을 찍고 붙이기

03 • 출력 정렬

가장 많은 종류부터 자동 정렬되어 출력

df 전체가 아니라 **한 열만** 골라낸 뒤 점을 찍는 순서 — 헷갈리면 오류

value_counts 기본 코드

한 열을 골라 점을 찍고 value_counts 붙이기

CODE

```
# 냉각기 상태별 사이클 건수
df["냉각기상태"].value_counts()
# 고장 40, 저하 40, 정상 40

# 밸브 상태별 사이클 건수
df["밸브상태"].value_counts()
# 정상 61, 지연 20, 경미 20, 심각 19
```

단 한 줄로 120행을 서너 개의 숫자로 요약 — 함께 따라 칠 코드는

개수를 비율로 바꾸기

개수 +비율 —둘을 함께 봐야 진짜 감이 잡힘

01 • 개수만의 한계

개수만으로는 많고 적음 판단 어려움

02 • 비율의 역할

전체에서 차지하는 비율로 봐야 비중이 보임

03 • 공정한 비교

규모가 다른 대상을 비교할 때 필수

1공장 불량 50건 vs 2공장 30건 — 생산량을 고려한 비율(1% vs 6%)이 실체를 보여줌

posco

normalize로 비율 구하기

value_counts에 normalize=True 옵션 추가

CODE

```
df["result"].value_counts(normalize=True).round(3)
# 정상 0.558
# 고장 0.442
# → 정상 약 55.8%, 고장 약 44.2%

# 개수와 비율을 나란히 보면 감이 잡힌다
df["result"].value_counts()   # 정상 67, 고장 53
```

정렬과 결측값 옵션

기본 동작을 바꾸고 싶을 때만 쓰는 두 가지 옵션

`sort=False`

개수 순서 대신
데이터 등장 순서로 표시

등급 A·B·C·D처럼 정해진 순서로 비교할 때 유용

`dropna=False`

기본은 빈 값 무시
결측도 하나의 종류로 셈

빈 칸의 개수도 중요한 정보일 때 사용

설비 데이터의 빈 칸은 센서 고장·통신 오류의 신호 — 어느 그룹에 몰렸는지가 단서

posco

실습 1. 값별 개수 세어 보기

범주 열의 값이 몇 번씩 나오는지 세어 데이터 구성 파악

목표

밸브 상태와 운전 부하의 값별 건수를 세어 표의 구성을 확인

단계

① 유압 사이클 파일을 읽어 전체 크기를 확인

② 밸브 상태 열의 값별 건수와 비율을 출력

③ 운전 부하 열도 같은 방법으로 세어 출력

④ 두 결과에서 가장 많은 값과 가장 적은 값을 각각 확인

예상 결과

밸브는 정상 61 • 지연 20 • 경미 20 • 심각 19 • 운전 부하는 60 대 60

실습 1 결과 해석

숫자를 읽고 데이터의 성격을 말해 보기

원리

세는 것은 한 열 단위

표 전체가 아니라 열 하나를 골라낸 뒤 센다.

결과는 값이 왼쪽, 건수가 오른쪽인 한 줄짜리 목록이다.

함정

출력 순서를 데이터 순서로 착각

기본은 건수가 많은 순서다.
정상·경미·지연·심각 순이 아니다.

정해진 순서로 보고 싶으면 정렬 옵션을 꺼야 한다.

검산

합이 120

밸브 61+20+20+19 = 120,
부하 60+60 = 120.

전체 행 수와 맞지 않으면 빈 칸이 있는 것이다.

세어 본 결과의 합이 전체 행 수와 맞는지 확인하는 습관을 들일 것

posco

실습 1 정답

한 열을 골라 값별 건수 세기

CODE

```
import pandas as pd

df = pd.read_csv('data/14_hydraulic.csv')
print(df.shape)                          # (120, 8)

print(df['밸브상태'].value_counts())
# 정상 61 · 지연 20 · 경미 20 · 심각 19

print(df['운전부하'].value_counts())
# 고부하 60 · 저부하 60

print(df['밸브상태'].value_counts().sum())   # 120
```

마지막 줄이 검산 — 세어 본 합이 전체 행 수와 같아야 한다

불균형 데이터의 의미

실제 제조 현장의 불량품 데이터가 대부분 띠는 모습 — 빈도 확인이 함정 회피의 첫 단추

제조 현장의 불량 데이터는 대부분 한쪽으로 심하게 쏠려 있음

쏠린 데이터에서는 「무조건 다수 쪽」이라 찍어도 정확도가 높게 나옴

빈도를 미리 확인하면 이 함정을 피할 수 있음

고장 예측 모델의 정확도에 속지 않으려면 —빈도 확인이 첫 안전장치

다음 — 수치형은 그냥 세면 안 된다, 구간으로 묶는 한 단계가 더 필요

실습 2. 비율로 치우침 확인하기

건수와 비율을 함께 구해 한쪽으로 심하게 치우친 데이터를 확인

목표

품질검사 표의 합격•불합격 건수와 비율을 구해 치우침의 정도를 확인

단계

① 품질검사 파일을 읽어 전체 크기를 확인

② 검사 결과 열의 값별 건수를 세어 출력

③ 같은 열의 비율을 구해 소수점 셋째 자리까지 정리해 출력

④ 불합격 비율을 백분율로 바꿔 말로 표현

예상 결과

합격 188 • 불합격 12 • 비율 0.94 대 0.06 — 불합격이 전체의 약 6%

실습 2 결과 해석

치우친 데이터가 왜 위험한지까지 말해 보기

원리

건수와 비율은 같이 봐야 한다

0.06 만으로는 감이 안 오고,
12건만으로는 비중을 모른다.

두 숫자를 나란히 놓아야 판단이 선다.

함정

정확도 94%에 속는 것

무조건 합격이라 찍는 엉터리 모델도 정확도가 94%다.

정작 찾고 싶은 불합격 12건은 하나도 못 잡는다.

검산

비율의 합이 1

0.94 + 0.06 = 1.00,
188 + 12 = 200.

합이 1이 안 되면 반올림 자리를 확인한다.

데이터를 받으면 가장 먼저 세어 본다 — 치우침은 나중에 발견할수록 비싸다

실습 2 정답

건수와 비율을 나란히 구해 치우침 확인

CODE

```
qc = pd.read_csv('data/14_hydraulic_qc.csv')
print(qc.shape)                          # (200, 11)

print(qc['검사결과'].value_counts())
# 합격 188 · 불합격 12

print(qc['검사결과'].value_counts(normalize=True).round(3))
# 합격 0.940 · 불합격 0.060

# 유압 사이클 쪽은 거의 반반 — 두 데이터의 성격이 다르다
print(df['result'].value_counts(normalize=True).round(3))
# 정상 0.558 · 고장 0.442
```

같은 코드로 두 데이터를 재어 보면 치우침이 무엇인지 바로 보인다

수치형을 구간으로 묶어 세기

먼저 묶고, 그 다음에 셈 — 순서가 핵심

STEP 1

pd.cut으로 구간 묶기

숫자를 정해진 경계에서 잘라 구간으로 묶음

예: 40도 미만 낮음 · 40~50 보통 · 50 이상 높음

→

STEP 2

value_counts 붙이기

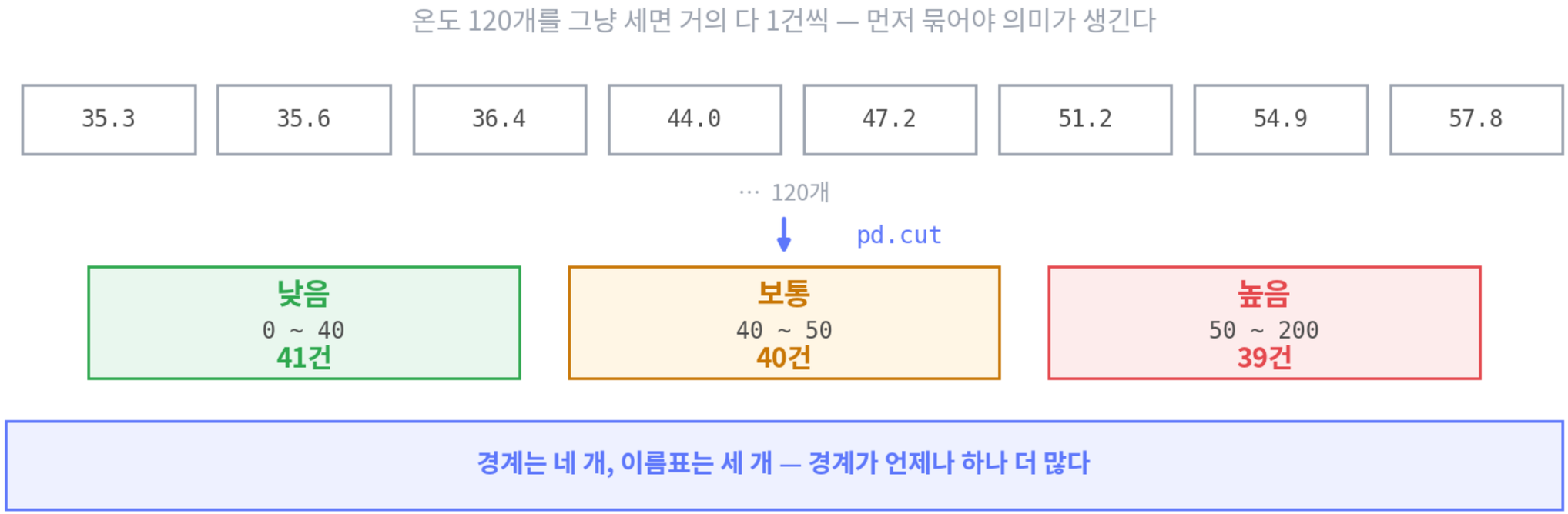
만든 구간 데이터에 붙여 구간별 빈도 세기

구간 → 개수의 두 단계 흐름

경계 기준은 설비의 정상 온도 범위 같은 현장 지식을 반영

그냥 세면 안 되는 열이 있다

수치형은 값이 거의 다 달라 그대로 세면 1건씩만 나옴 — 먼저 묶어야 의미가 생김



구간으로 묶은 뒤 세면 분포가 보인다. 경계를 어디에 둘지가 실력이다.

경계는 현장 지식으로 정한다 — 아무 데나 자르면 아무 뜻 없는 구간이 나온다

pd.cut 구간 빈도 코드

구간으로 묶은 뒤 value_counts로 세기

CODE

```
band = pd.cut(df["온도"],
              bins=[0, 40, 50, 200],
              labels=["낮음", "보통", "높음"])
band.value_counts()
# 낮음 41, 보통 40, 높음 39
```

경계 4개면 구간 3개 — 경계가 이름표보다 하나 많다는 점 주의 · 함께 따라 칠 코드는

실습 3. 구간으로 묶어 세어 보기

수치형 열을 구간으로 나눈 뒤 구간별 건수를 세어 분포 확인

목표

진동 값의 범위를 확인하고 세 구간으로 묶어 어디에 몰려 있는지 파악

단계

① 유압 사이클 파일에서 진동 열의 최솟값과 최댓값을 확인

② 그 범위를 감안해 경계 네 개와 이름표 세 개를 정해 구간으로 묶기

③ 묶은 결과의 구간별 건수를 세어 출력

④ 가장 많은 구간과 가장 적은 구간이 무엇인지 확인

예상 결과

진동 범위는 0.53 ~ 0.779 · 보통 55 · 약함 48 · 강함 17

실습 3 결과 해석

구간을 나눌 때 반드시 확인할 세 가지

원리

묶기와 세기는 다른 단계

구간으로 묶는 것과 세는 것은 별개다.

묶기만 하면 값 대신 구간 이름이 붙은 목록이 나올 뿐 건수는 안 나온다.

함정

경계가 이름표보다 하나 많아야 함

이름표 셋에 경계를 셋만 적으면 오류가 난다.

경계 사이의 칸이 구간이므로 언제나 하나 더 많다.

검산

합이 120

55 + 48 + 17 = 120.

합이 모자라면 경계 밖으로 빠져나간 값이 있는 것이다.

합이 전체 행 수보다 적으면 경계 범위를 벗어난 값이 있다는 신호

posco

실습 3 정답

범위를 먼저 확인하고 세 구간으로 묶어 세기

CODE

```
print(df['진동'].min(), df['진동'].max())   # 0.53 0.779

band = pd.cut(df['진동'],
              bins=[0, 0.6, 0.7, 10],
              labels=['약함', '보통', '강함'])

print(band.value_counts())
# 보통 55 · 약함 48 · 강함 17

print(band.value_counts().sum())      # 120
```

경계 넷에 이름표 셋 — 맨 끝 10은 실제 최댓값보다 넉넉하게 잡은 값

02

그룹 집계

분할 · 적용 · 결합 — 기준을 정해 나눠 보기

그룹으로 나눠 보는 이유

전체 평균 하나로는 어느 상태가 문제인지 알 수 없음

전체 평균 하나로는 그룹 사이의 차이가 묻힘

의미 있는 그룹으로 나눈 뒤 그룹마다 따로 집계

문제가 집중된 곳을 빠르게 찾아내는 도구

얼음물·끓는물에 한 발씩 담근 사람의 평균 체감 온도가 '적당함'이라 나오는 이유

유압 전체 평균 온도 45.34도 — 그런데 냉각기 정상은 35.89, 고장은 54.67

분할-적용-결합 구조

판다스의 그룹 집계는 항상 이 3단계로 작동

원본

전체 데이터

그룹으로 나누기 전 상태

→

01 • 분할

그룹별로 나눔

냉각기상태 기준이면 정상·저하·고장 세 덩어리, 아직 계산 없음

→

02 • 적용

각 그룹 계산

정상의 평균, 저하의 평균, 고장의 평균을 각각 계산

→

03 • 결합

결과 합침

각 그룹 결과를 하나의 표로 — 우리가 보는 최종 결과

적용 단계의 계산만 갈아 끼우면 평균·합계·최댓값 모두 구할 수 있음 — 분할은 그대로

groupby 기본 사용법

왼쪽에서 오른쪽으로 — 세 부분으로 끊어 읽기

01 · 그룹 기준 정하기

groupby 괄호 안에 — 줄을 나눌 열

02 · 집계할 열 고르기

[ ] 대괄호로 — 숫자로 답할 열

03 · 집계 함수 붙이기

.mean() 등 — 어떻게 계산할지

"df를 가지고, 냉각기상태으로 그룹을 나눈 뒤, 온도 열을 골라서, 평균을 내라" — 문장처럼 끊어 읽기

posco

groupby 기본 코드

기준 → 열 → 함수 순서로 붙이기

CODE

```
df.groupby("냉각기상태")["온도"].mean().round(2)
# 냉각기상태
# 고장 54.67
# 저하 45.46
# 정상 35.89

# 함수만 갈아 끼우면 다른 통계도 그대로
df.groupby("냉각기상태")["온도"].max()    # 57.8 / 51.2 / 36.4
```

고장 54.67, 정상 35.89 — 18.78도 차이가 한 줄로 드러남

실습 4. 그룹으로 나눠 재어 보기

기준을 정해 나눈 뒤 그룹마다 통계를 내 차이를 확인

목표

냉각기 상태별로 온도의 평균과 최댓값을 구해 세 상태가 얼마나 다른지 확인

단계

① 유압 사이클 파일을 읽어 냉각기 상태별 사이클 수를 세기

② 냉각기 상태로 그룹을 나눠 온도의 평균을 소수점 둘째 자리까지 구하기

③ 같은 기준으로 최댓값도 구해 평균과 나란히 비교하기

④ 가장 뜨거운 상태와 가장 시원한 상태의 평균 차이를 계산하기

예상 결과

세 상태 모두 40건씩 • 평균 54.67 • 45.46 • 35.89 • 차이 18.78도

실습 4 결과 해석

전체 평균 하나로는 왜 안 되는지 숫자로 확인

원리

기준 • 열 • 함수
세 부분으로 읽는다

groupby 괄호에 나눌 기준, 대괄호에 계산할 열, 마지막에 어떻게 계산할지.

함수만 갈아 끼우면 평균도 최댓값도 같은 자리에서 나온다.

함정

전체 평균 하나로
판단하는 것

전체 평균은 45.34다. 세 상태 어디에도 해당하지 않는 숫자다.

나누기 전에는 문제가 어디 있는지 보이지 않는다.

검산

40 • 40 • 40

세 그룹의 건수가 모두 40이고 합이 120이면 맞다.

평균 차이는 54.67 − 35.89 = 18.78 이다.

그룹으로 나누는 순간 전체 평균이 감추고 있던 18.78도의 차이가 드러난다

실습 4정답

기준 → 열 → 함수, 함수만 갈아 끼우기

CODE

```
import pandas as pd
df = pd.read_csv('data/14_hydraulic.csv')

print(df.groupby('냉각기상태').size())
# 고장 40 · 저하 40 · 정상 40      세 그룹이 고르게 나뉨

print(df.groupby('냉각기상태')['온도'].mean().round(2))
# 고장 54.67 · 저하 45.46 · 정상 35.89

print(df.groupby('냉각기상태')['온도'].max())     # 함수만 갈아 끼움
# 고장 57.8 · 저하 51.2 · 정상 36.4

print(round(54.67 - 35.89, 2))        # 18.78
print(round(df['온도'].mean(), 2))     # 45.34   전체 평균
```

정상의 최댓값 36.4가 고장의 평균 54.67보다 낮다 — 두 상태는 아예 겹치지 않는다

여러 기준으로 그룹 묶기

고장이면서 고부하처럼 — 두 기준을 겹쳐 더 잘게 나눠 보기

01 · 여러 기준 묶기

그룹 기준 두 개 이상을 대괄호로 묶기

02 · 더 세밀한 그룹

기준들을 조합한 세밀한 그룹 생성

03 · 순서가 관점

기준 순서에 따라 보는 관점이 달라짐

냉각기 먼저면 상태별로 보이고, 부하 먼저면 부하별로 보임 — 목적에 맞게 순서 결정

posco

여러 키 그룹 코드

기준 여러 개를 대괄호로 묶기

CODE

```
df.groupby(["냉각기상태", "운전부하"])["온도"].mean().round(2)
# 고장  고부하  55.51
# 고장  저부하  54.05
# 저하  고부하  44.07
# 저하  저부하  45.58
# 정상  고부하  35.89

df.groupby(["냉각기상태", "운전부하"]).size()
# 17 / 23 / 3 / 37 / 40   ← 여섯 줄이 아니라 다섯 줄
```

3 × 2 = 6 조합인데 다섯 줄 — 정상이면서 저부하인 사이클이 한 건도 없음

두 기준 결과를 가로세로 표로 — crosstab

같은 숫자인데 세로로 길게 보느냐, 가로세로 표로 보느냐

구분	groupby 두 기준	crosstab
모양	세로로 길게 5줄	가로세로 3 × 2 표
0인 조합	줄이 없어 안 보임	칸에 0으로 찍힘
비율	직접 나눠야 함	normalize='index' 한 줄

엑셀 피벗테이블과 같은 모양 — 두 범주의 관계를 볼 때 실무에서 가장 많이 쓰는 형태

posco

crosstab • unstack 코드

빈도는 crosstab 으로, 이미 만든 groupby 결과는 unstack 으로

CODE

```
pd.crosstab(df["냉각기상태"], df["운전부하"])
#          고부하  저부하
# 고장       17     23
# 저하        3     37
# 정상       40      0     <- 앞 장 목록에는 아예 없던 줄

pd.crosstab(df["밸브상태"], df["result"], normalize="index").round(3)
# 심각    0.737  0.263    <- 행마다 합이 1 이 되는 비율
# 정상    0.197  0.803

df.groupby(["냉각기상태", "운전부하"])["온도"].mean().round(2).unstack()
#          고부하  저부하
# 고장     55.51  54.05
# 정상     35.89    NaN   <- 없는 조합은 빈 칸
```

crosstab 은 세면서 표로 만들고, unstack 은 이미 만든 결과를 표로 편다 — 모양은 같다

size와 count 차이

핵심 차이는 결측값 처리

구분	size	count
대상	전체 행	비결측 값
결측값	포함	제외
열 선택	불필요	필요

빈 칸이 없으면 둘이 같음 · 이 데이터는 결측이 없어 size 와 count 가 항상 일치

03

종합 실습

빈도와 그룹 집계를 한 줄로 이어 우선 점검 대상 찾기

실습 5. 빈도와 그룹을 이어 보기

구성을 세어 보고, 그룹으로 나눠 어느 상태를 먼저 점검할지 정하기

목표

밸브 상태별 건수와 고장 건수를 함께 구해 우선 점검 대상을 고르기

단계

① 유압 사이클 파일을 읽어 밸브 상태별 건수를 세기

② 판정이 고장인 사이클만 걸러 새 변수에 담기

③ 걸러낸 결과를 밸브 상태별로 나눠 건수를 세기

④ 두 결과를 나눠 상태별 고장 비율을 구하고 가장 높은 상태를 확인

예상 결과

밸브 심각은 19건 중 14건이 고장 — 비율 0.737 로 가장 높음

실습 5 결과 해석

건수와 비율이 서로 다른 답을 줄 때

원리

두 목록을 나누면 판다스가 짝을 맞춘다

고장 건수 목록을 전체 건수 목록으로 나누면
밸브 이름이 같은 것끼리 짝을 지어 계산된다.

순서가 달라도 이름으로 맞춰 준다.

함정

그룹 크기를 안 보고 건수만 비교

심각은 12건과 비슷한 14건이지만
모수가 19건뿐이다.

비율 73.7% — 네 배 가까이 위험하다.

검산

12+13+14+14 = 53

밸브 상태별 고장 건수의 합이
전체 고장 53건과 같아야 한다.

안 맞으면 거르는 조건을 확인한다.

다음 시간에서는 — 그룹별 비교를 평균을 넘어 흩어짐과 여러 통계 묶음까지 확장

실습 5 정답

세고 · 거르고 · 묶은 뒤, 두 목록을 나눠 비율로

CODE

```
df = pd.read_csv('data/14_hydraulic.csv')

전체 = df['밸브상태'].value_counts()
print(전체)          # 정상 61 · 지연 20 · 경미 20 · 심각 19

고장 = df[df['result'] == '고장']
상태별 = 고장.groupby('밸브상태').size()
print(상태별)        # 경미 14 · 심각 14 · 정상 12 · 지연 13
print(상태별.sum())  # 53

비율 = (상태별 / 전체).round(3)
print(비율.sort_values(ascending=False))
# 심각 0.737 · 경미 0.700 · 지연 0.650 · 정상 0.197
```

건수는 비슷한데 비율은 네 배 차이 — 모수를 함께 봐야 판단이 선다

실습 — 종합 산출물

세 문장으로 정리 — 다음 시간 리포트의 기초

01 · 전체 빈도

전체 빈도에서
알게 된 것 1가지

예: "불합격은 전체의 약 6%, 합격이 압도적"

02 · 그룹 비교

그룹 비교에서
알게 된 것 1가지

예: "밸브가 심각인 사이클은 열에 일곱이 고장으로 기록됨"

03 · 더 볼 것

더 살펴봐야 할 그룹
1가지와 이유

다음 분석의 출발점 — 가설을 다음 단계로